

Структура в ОЗУ

Для управления Phoenix DB и обеспечения работы Flash-Friendly алгоритма, используется следующая структура:

- `int lastBlockId`
 - Последний использованный Id блока. Для создания новых блоков.
- `long lastBlockVersion`
 - Последняя использованная версия блока. Используется при формировании новых версий / обновлений
- `int lastBlockNumber`
 - Номер блока (позиция) последнего обновленного блока. Данное значение имеет ключевое значение в процессе обновления.
- `TreeMap<Integer, Integer> occupiedBlockNumbers`
 - Номера блоков (позиции), занятые активными версиями блоков. Используется при обновлениях.
- `Map<Integer, BlockNumberAndVersion> blockInfos`
 - Краткое видео по активным блокам. Содержит версию блока, номер блока (позицию) и количество копий (по Id блока).
- `Map<Integer, Integer> blockIdByBlockNumber`
 - Матрица номеров блоков (позиций) по Id блоков. Необходимо при обновлении для идентификации блоков.
- `Set<Integer> tombstonedBlockIds`
 - Содержит идентификаторы Tombstoned-блоков.

Загрузка получается следующей:

- 1) Заполнить `blockIdByBlockNumber`
- 2) Параллельно обновлять `blockInfos`
- 3) Идентифицировать Tombstoned-блоки с 1 копией, а в более чем одной проиндексировать в `tombstonedBlockIds`
- 4) Затем сформировать `occupiedBlockNumbers` из `blockInfos`

Алгоритм записи изменений

- 1) Граница - это блок с макс вершей
- 2) Необходимо найти 2 блока
 - $b1$ - блок с макс вершей "справа" от границы
 - $b2$ - блок с макс вершей "слева" от границы
- 3) Также обозначали блок непосредственно справа от границы как $b3$
Если существует обновленный блок, обозначали его, как $b0$

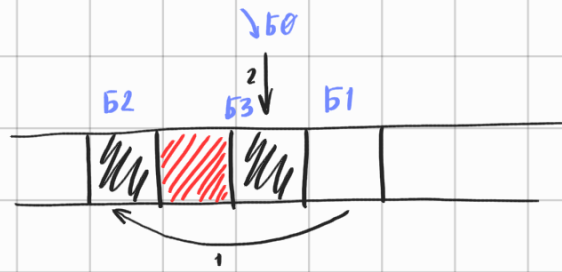
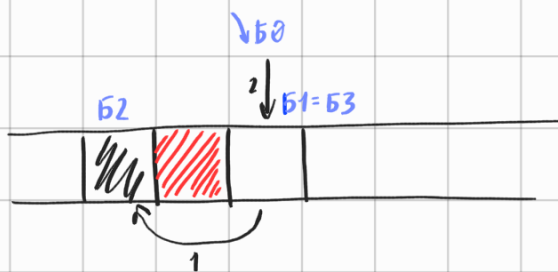
4) Пошагово в ситуации, когда более 1 блока свободно

4.1) Перемещали $b1$ в $b2$

- освобождали $b1$

4.2) Записывали обновление в $b3$

- если есть $b0$, освобождали $b0$



5) Пошагово в ситуации, когда свободно всего 1 блок

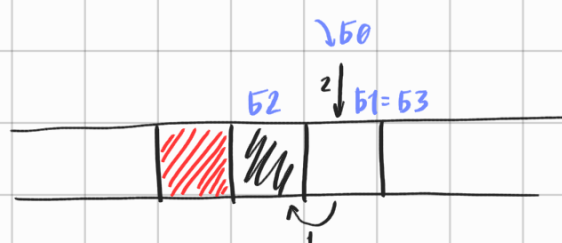
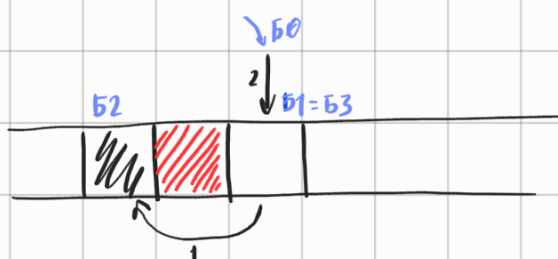
5.1) Перемещали $b1$ в $b2$

- освобождали $b1$

5.2) Записывали обновление в $b1$

- освобождали все неиспользуемые блоки, не имеющие копий.

- При этом, если не существует неиспользуемых блоков, добавление нового блока невозможно, а возможно только обновление



Подробнее

4.1) Б2 - был свободным

- удалили номер блока Б2 из `BlockIdByBlockNumber`
- инкрементировали список копий связанного блока Б2-Id (в `BlockInfos` по Id)
- если копий теперь ≤ 1 , и блок Б2 ТБ, деактивировали блок удалили из `BlockInfos` и `tombstonedBlockIds` Б2-Id
- удалили Б2 номер блока из `occupiedBlockNumbers`

Б1 - новое вернее переименовать на Б2

- добавили номер блока Б2 в `BlockIdByBlockNumber` с Б1-id
- если Б1 `Tombstoned`, добавили в `tombstonedBlockIds` Б1-id
- инкрементировали список копий в `BlockInfos` где Б1-id и также обновляли версию, номер и `Tombstoned` статус
- добавили номер блока Б2 в `occupiedBlockNumbers`
- Б1 теперь старый, удалили номер блока Б1 из `occupiedBlockNumbers`

4.2) Б3 - то же, что и Б2

Если Б0 был, то же, что и Б1, только переименовать на Б3

Если Б0 не было (новый блок создан на позиции номер Б3)

- добавили номер блока Б3 в `BlockIdByBlockNumber` с Б0-id
- если Б0 `Tombstoned`, добавили в `tombstonedBlockIds` Б1-id
- создали новую запись в `BlockInfos`
кол-во копий 1, номер блока Б3, версия и TS-статус
- добавили номер блока Б3 в `occupiedBlockNumbers`